

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

6) Ans

Problem Breakdown

- * Successful Phases: requirement and design
- * Failure Points: implementation (where the actual coding occurs)
- * ~~consequence~~ The ~~failure~~ in implementation caused a "ripple effect", leading to failures in the Testing.

Key insights

- * Error Propagation: Because the foundation was faulty, downstream could not succeed.
- * Identification: A concept map was used to pinpoint implementation as the root
- * Methodology: Layer-wise troubleshooting was employed to isolate the specific "layer" of the SDLC.

ANS

UML diagram Types:

- * Use Case diagram: Focuses on user interaction.
- * Class diagram: Defines the structure of the system.
- * Sequence diagram: Illustrates the interaction flow over time.
- * Activity: Maps out the workflow or operational steps.

Analysis:

- * Clarity: Helps developers understand the system clearly.
- * Communication: Bridges the gap between different stakeholders.
- * Methodology: Directly supports object-oriented programming.

4) Ans

Core Concepts:

The relationship between modularity, the single responsibility principle, open/closed principle.

- * Modularity: Breaking a system into smaller manageable parts.
- * SRP: Limiting each class to a single responsibility.
- * OCP: Allowing code to be extended without changing existing source code.

System Benefits:

How these principles work together to reduce complexity, improve maintainability and enhance scalability in software system.

Ans

a) Association:

- * Nature: General relationship
- * Detail: Basic connection between objects.

b) Aggregation:

- * Nature: "Has-a" relationship
- * Detail: Weak ownership (object can exist)

c) Composition:

- * Nature: Strong ownership
- * Detail: Dependent existence

Impact & Purpose

- * Intersection: Defines how object interact
- * Structure: Ensures proper system organization
- * Design: Improves reusability and flexibility

2)

Ans

Phase Transformation Each Phase acts as a stepping stone to the next transformation of the system.

- * Required: These are gathered and then converted into a design.
- * Design: This architecture is then converted into code during the implementation.
- * Code: The resulting software is tested for bugs and then developed for user access.
- * Structural Development: Ensures a clear roadmap and prevent chaos during the building.
- * Modular Building: Supports breaking the system into manageable pieces, making complex projects easier to handle.

Ques

OOP Concept in the sequence

Key definitions

- * Encapsulation: The process of bundling data and the methods that operate on the data into a single unit.
- * Abstraction: The practice of hiding complex implementation details and showing only the essential features.
- * Concept - Primary benefit.
- * Encapsulation - Enhances data security.
- * Abstraction - Increases simplicity by filtering out unnecessary details.
- * Combined - Result in modular and maintainable software system.

(a) Explain structured design approach in software development.

- Dividing system into modules

(b) Compare structured approach with object-oriented approach.

- Structured: Function-based
- OO: Object-based

(c) Explain how modular design improves system efficiency and maintenance.

- Easy debugging, scalability, and reuse

- iv. Testing
- v. Deployment
- vi. Maintenance

(b) Describe how each phase contributes to structured development.

- Ensures proper planning and execution

(c) Explain the importance of SDLC in delivering reliable software.

- Reduces risk and improves quality

4. Consider a project where requirements are clearly defined from the beginning.

(a) Explain different SDLC models:

- i. Waterfall Model
- ii. Iterative Model
- iii. Spiral Model
- iv. Agile Model

(b) Compare Waterfall and Agile models.

- Waterfall: Sequential, rigid
- Agile: Flexible, iterative

(c) Suggest a suitable model for dynamic requirements and justify.

- Agile model is suitable due to adaptability

5. Suppose a system analyst is designing an ATM System.

(a) Explain the importance of Unified Modeling Language (UML) in OOAD.

- Visual representation of system

(b) Describe common UML diagrams:

- i. Use Case Diagram
- ii. Class Diagram
- iii. Sequence Diagram
- iv. Activity Diagram

(c) Explain how UML supports object-oriented methodologies.

- Helps in analysis, design, and communication

6. Consider a scenario where a system needs to be designed for Hospital Management.

Annexure A

Short Answer Test

1. Suppose a software system is developed for a **Library Management System** where users can search books, borrow, and return them.

(a) Explain the fundamental object-oriented concepts such as **class**, **object**, **encapsulation**, **inheritance**, and **polymorphism** in this context.

- i. **Class**: Blueprint (e.g., Book, User)
- ii. **Object**: Instance of class
- iii. **Encapsulation**: Data hiding using methods
- iv. **Inheritance**: Reusability (e.g., Student inherits User)
- v. **Polymorphism**: Same method, different behavior

(b) Describe different types of **object relationships** such as **association**, **aggregation**, and **composition**.

- i. **Association**: General connection
- ii. **Aggregation**: Weak "has-a" relationship
- iii. **Composition**: Strong ownership

(c) Illustrate how these concepts help in building a modular and reusable system.

- Improves maintainability, flexibility, and scalability

2. Consider developing a **Student Information System**.

(a) Explain key object-oriented design principles such as:

- i. **Abstraction** – Showing essential features
- ii. **Encapsulation** – Protecting data
- iii. **Modularity** – Dividing into smaller components

(b) Describe principles like **Single Responsibility Principle (SRP)** and **Open/Closed Principle (OCP)**.

- i. **SRP**: One class → one responsibility
- ii. **OCP**: Open for extension, closed for modification

(c) Explain how these principles improve software quality.

- Reduces complexity, improves code reuse and maintainability

3. Suppose a company plans to develop an **Online Shopping System**.

(a) Explain the phases of **SDLC**:

- i. **Requirement Analysis**
- ii. **Design**
- iii. **Implementation**

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I 192511886/ANNEKOR (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: